ERP DEVELOPMENT ROADMAP

Complete Feature Status & Action Plan

Last Updated: February 2026

System: Multi-tenant ERP (Stock, Accounting, HR)

TABLE OF CONTENTS

---

1. SYSTEM ARCHITECTURE OVERVIEW

Tech Stack

Layer	Technology
Framework	Next.js 16 (App Router, Turbopack)
Database	MongoDB + Mongoose
Auth	NextAuth.js v5
Styling	Tailwind CSS + OKLCH Colors
Charts	Recharts
PDF	@react-pdf/renderer v4.3.0
Language	JavaScript + TypeScript (mixed)

Design System

Element	Value
Primary Color	Yellow (#eab308)
Font	Geist Sans / Geist Mono
Theme	Light/Dark with OKLCH
Style	GitHub/Vercel inspired
Mobile	Cards view, icon-only buttons
Desktop	Table view, labeled buttons

Role Hierarchy

```
SuperAdmin (Platform-wide)
  └── Admin (Company-level)
        └── Manager
              └── Accountant
              └── Store Manager
                    └── Employee
```

Multi-Tenancy Architecture

The system uses a **shared database, schema-isolated** approach where all tenants share the same MongoDB database but data is isolated using `companyId` scoping.

Core Tenant Utilities (`/lib/utils/tenant-utils.js`)

Function	Purpose	Usage
`getTenantContext()`	Get companyId, companyCode, isSuperAdmin from session	Every query/action
`withTenantScope(query, companyId, isSuperAdmin)`	Add companyId to query filter	Find operations
`tenantFilter(companyId, isSuperAdmin)`	Build base tenant filter	Simple queries
`withTenantPipeline(pipeline, companyId, isSuperAdmin)`	Prepend $match to aggregation	Aggregations
`validateTenantAccess(doc, companyId, isSuperAdmin)`	Check document belongs to tenant	Before update/delete
`getCompanyIdForCreate(explicit, userCompanyId, isSuperAdmin)`	Get companyId for new documents	Create operations

Schema-Level Isolation Pattern

Every tenant-scoped schema follows this pattern:

```
// 1. companyId field (required, indexed)
companyId: {
  type: Schema.Types.ObjectId,
  ref: "Company",
  required: [true, "Company ID is required"],
  index: true,
},

// 2. Compound indexes always include companyId first
schema.index({ companyId: 1, status: 1, createdAt: -1 });
schema.index({ companyId: 1, documentNumber: 1 }, { unique: true });

// 3. Static methods require companyId parameter
schema.statics.getByType = function(companyId, type) {
  if (!companyId) throw new Error("companyId required for tenant
isolation");
  return this.find({ companyId, type });
};
```

Role-Based Data Access

Role	companyId	Data Access
SuperAdmin	null	Platform-wide (all companies)
Admin	set	Company-scoped only
Manager/Accountant	set	Company-scoped only
Employee	set	Company-scoped only

Multi-Tenancy: Tax Module

Schema: `TaxTransaction` (taxTransactions.js)

Isolation Layer	Implementation
Schema Field	`companyId` (required, indexed)
Unique Constraint	`(companyId, transactionNumber)`
Query Indexes	`(companyId, transactionDate)`, `(companyId, taxType)`, `(companyId, filingPeriod)`

Static Methods (all require companyId):

```
TaxTransaction.getByType(companyId, taxType, startDate, endDate)
TaxTransaction.getUnfiled(companyId, taxType)
TaxTransaction.getUnremittedWHT(companyId)
TaxTransaction.getVATReturn(companyId, filingPeriod)
TaxTransaction.getWHTReportByRate(companyId, startDate, endDate)
TaxTransaction.getWHTReportByParty(companyId, startDate, endDate)
```

Query Pattern (taxQueries.js):

```
const { companyId, isSuperAdmin } = await getTenantContext();
const tenantMatch = isSuperAdmin ? {} : { companyId: new
ObjectId(companyId) };
// All aggregations and queries use tenantMatch
```

Supported Tax Types (Kenya Compliance):

- VAT: `vat_input`, `vat_output`
- WHT: `wht`, `wht_received`
- Payroll: `paye`, `nssf`, `nhif`, `housing_levy`
- Other: `excise_duty`, `advance_tax`, `dst`, `turnover_tax`, `cgt`

Multi-Tenancy: Bank Feed Module

Schemas: `BankStatement`, `BankFeedLine` (bankFeed.js)

Schema	Isolation Fields	Unique Constraints
BankStatement	`companyId`, `bankAccountId`	`(companyId, contentHash)`
BankFeedLine	`companyId`, `statementId`, `bankAccountId`	`(companyId, lineHash)`

Indexes:

```
// BankStatement
{ companyId: 1, createdAt: -1 }
{ companyId: 1, bankAccountId: 1, status: 1 }
{ companyId: 1, contentHash: 1 }  // unique - prevent duplicate uploads

// BankFeedLine
{ companyId: 1, status: 1, transactionDate: -1 }
{ companyId: 1, lineHash: 1 }  // unique - prevent duplicate transactions
```

Allocation Flow (tenant-scoped):

1. **Import:** Statement created with `companyId` from session
2. **Parse:** Lines inherit same `companyId`
3. **Match:** Suggestions query only invoices/bills for same `companyId`
4. **Allocate:** Journal entry created with `companyId`

Allocation Types:

- `invoice_payment` - Match to customer invoice
- `bill_payment` - Match to vendor bill
- `expense` - Direct expense allocation
- `income` - Direct income allocation
- `transfer` - Inter-bank transfer
- `split` - Split across multiple accounts

Multi-Tenancy: Reports Module

Service: `ReportService` (reportsService.js)

Pattern: Every report method gets tenant context first:

```
static async generateProfitLoss(startDate, endDate) {
  const { companyId, isSuperAdmin } = await getTenantContext();

  // All account queries are tenant-scoped
  const accounts = await Account.find(
    withTenantScope(
```

```
        { accountType: { $in: ["revenue", "expense"] } },
        companyId,
        isSuperAdmin
      )
    ).lean();

    // Balance calculations pass companyId
    const balances = await this.calculateAccountBalances(
      accounts, startDate, endDate, companyId, isSuperAdmin
    );
    // ...
  }
```

Reports Available (all tenant-scoped):

Report	Query Location	Tenant Method
Profit & Loss	ReportService	`withTenantScope()`
Balance Sheet	ReportService	`withTenantScope()`
Trial Balance	ReportService	`withTenantScope()`
Cash Flow	ReportService	`withTenantScope()`
General Ledger	ReportService	`withTenantScope()`
AR/AP Aging	JournalEntry statics	`companyId` param
VAT Return	TaxTransaction statics	`companyId` param
WHT Report	TaxTransaction statics	`companyId` param

SuperAdmin Platform Reports:

- Company-level metrics across all tenants (aggregate only)
- Platform activity summaries
- Cross-tenant statistics (no detail data exposed)

---

2. CURRENT FEATURE STATUS

Complete Features ✅

Feature	Schema	Actions	UI	Queries	PDF	Notes
Products	100%	95%	95%	90%	-	Full costing/inventory
Stock Movements	100%	95%	90%	95%	-	Immutable + accounting
Item Checkouts	100%	95%	90%	90%	-	Loan tracking
Stock Requests	100%	95%	90%	85%	-	Multi-fulfillment
Stock Adjustments	100%	100%	85%	80%	-	Auto journal entries

Feature	Schema	Actions	UI	Queries	PDF	Notes
Invoices	100%	95%	95%	95%	✅	Full COGS + revenue
Purchase Orders	100%	95%	95%	90%	✅	Complete workflow
Quotes	100%	95%	95%	90%	✅	Convert to invoice
Payments	100%	95%	90%	85%	-	Made + Received
Bills	95%	95%	90%	95%	-	Approval + WHT
Journal Entries	100%	90%	85%	95%	-	Double-entry
Chart of Accounts	100%	95%	90%	85%	-	Kenya standard
Parties	100%	90%	85%	90%	-	Customer/Supplier/Employee
Fiscal Periods	100%	95%	95%	90%	-	**NEW** Period close/lock UI
Tax Transactions	100%	75%	60%	85%	-	VAT + WHT tracking
Employee Claims	95%	90%	85%	85%	-	Advance + Reimbursement
Categories	100%	95%	90%	85%	-	Product categories
Company Management	100%	95%	90%	95%	-	Multi-tenant
User Management	100%	90%	90%	85%	-	Roles + linking

Partially Complete Features 🔨

Feature	Schema	Actions	UI	Queries	Priority	Notes
Expenses	90%	60%	85%	90%	🟡 MED	Approval workflow incomplete
Credit Notes	95%	50%	80%	80%	🟡 MED	Core works, edge cases
Bank Feed	80%	70%	60%	90%	🔴 HIGH	**NEW** Reconciliation WIP
Delivery Notes	80%	70%	60%	85%	🟢 LOW	Basic functionality

Dashboard Status

Dashboard	Layout	KPIs	Charts	Lists	Alerts
SuperAdmin	80%	85%	80%	70%	60%
Admin	85%	90%	85%	80%	70%
Accountant	50%	60%	50%	40%	40%
Store Manager	75%	80%	70%	70%	60%
Employee	80%	85%	60%	80%	70%
Executive	30%	40%	30%	20%	20%

Reports Status

Report	Query	UI	Export	Status
Balance Sheet	90%	85%	-	🟢 Ready
Profit & Loss	90%	85%	-	🟢 Ready
Trial Balance	85%	80%	-	🟢 Ready
General Ledger	85%	80%	-	🟢 Ready
AR Aging	90%	85%	-	🟢 Ready
AP Aging	90%	85%	-	🟢 Ready
Sales Report	85%	80%	-	🟢 Ready
Inventory Report	80%	75%	-	🟡 WIP
Cash Flow	60%	50%	-	🟡 WIP
VAT Report	70%	60%	-	🟡 WIP
WHT Report	70%	60%	-	🟡 WIP

---

3. SCHEMA INVENTORY

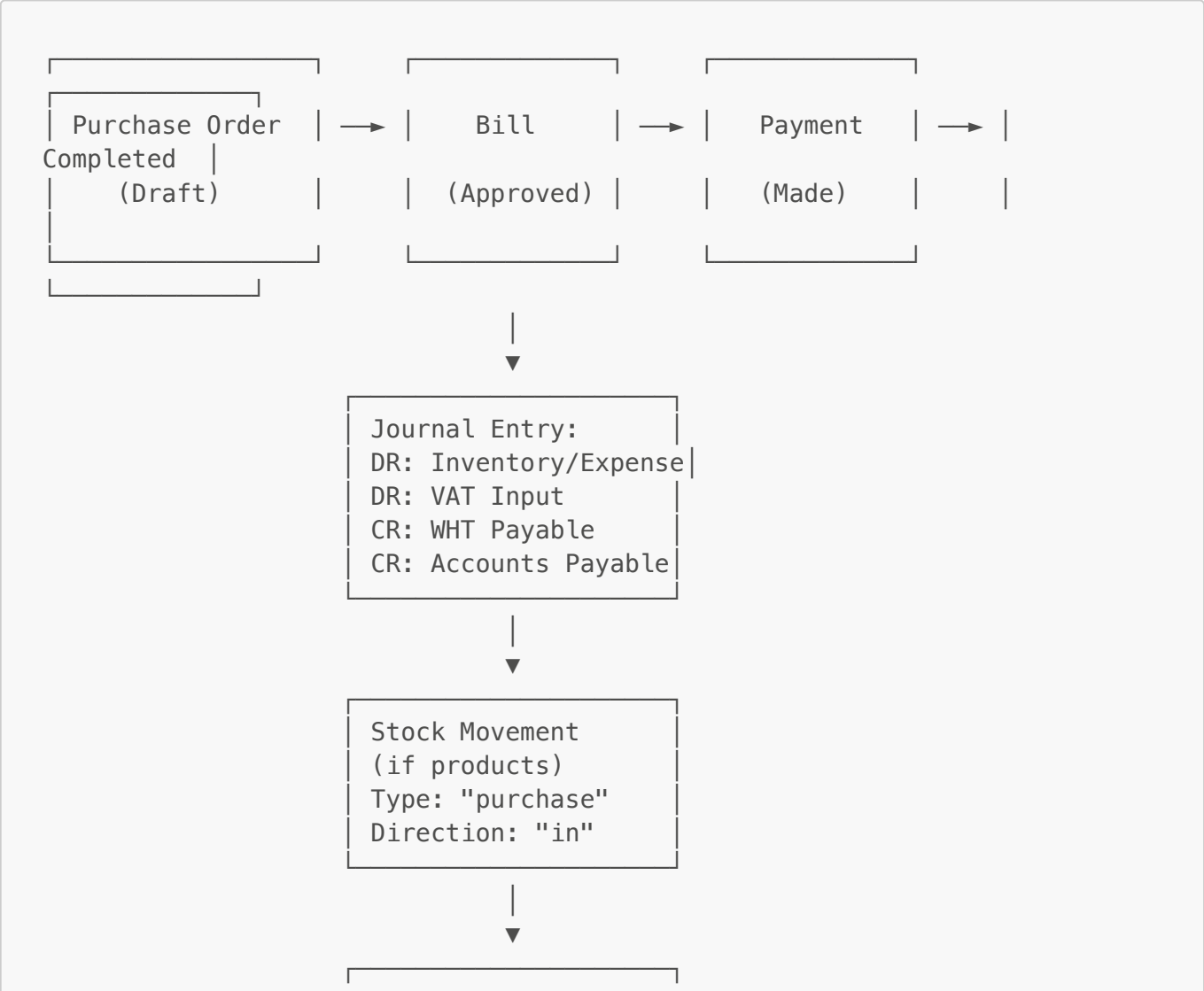
Complete Schemas (Location: `/app/models/`)

Schema	File	Size	JE Integration	Status
Product	`product.js`	24KB	Via Bill/Invoice	✅ Complete
StockMovement	`stockmovement.js`	15KB	✅ Yes	✅ Complete
StockRequest	`requests.js`	12KB	Via fulfillment	✅ Complete
ItemCheckout	`checkouts.js`	10KB	Via return	✅ Complete
InventoryAdjustment	`inventoryAdjustment.js`	8KB	✅ Auto	✅ Complete
Invoice	`invoice.js`	44KB	✅ Revenue + COGS	✅ Complete
Bill	`bill.js`	34KB	✅ AP + Inventory	✅ Complete
Quote	`quote.js`	25KB	N/A	✅ Complete
PurchaseOrder	`purchaseOrder.js`	27KB	Via Bill	✅ Complete
Payment	`payment.js`	27KB	✅ Yes	✅ Complete
CreditNote	`creditNote.js`	19KB	✅ Yes	✅ Complete
Expense	`expenses.js`	17KB	✅ Yes	🔨 90%
EmployeeClaim	`employeesClaims.js`	17KB	✅ Yes	✅ Complete

Schema	File	Size	JE Integration	Status
JournalEntry	JournalEntry.js	19KB	N/A (is JE)	✅ Complete
Account	account.js	12KB	N/A	✅ Complete
Party	parties.js	10KB	Via JE party	✅ Complete
FiscalPeriod	fiscalPeriod.js	22KB	Should validate	✅ Complete
TaxTransaction	taxTransactions.js	25KB	Links to JE	✅ Complete
DeliveryNote	dnote.js	8KB	❌ No	🔨 80%
Company	Company.js	15KB	N/A	✅ Complete
User	user.js	8KB	N/A	✅ Complete
BankFeed	bankFeed.js	8KB	Via allocation	🔨 80%
Counter/ErpCounter	erp-counter.js	5KB	N/A	✅ Complete

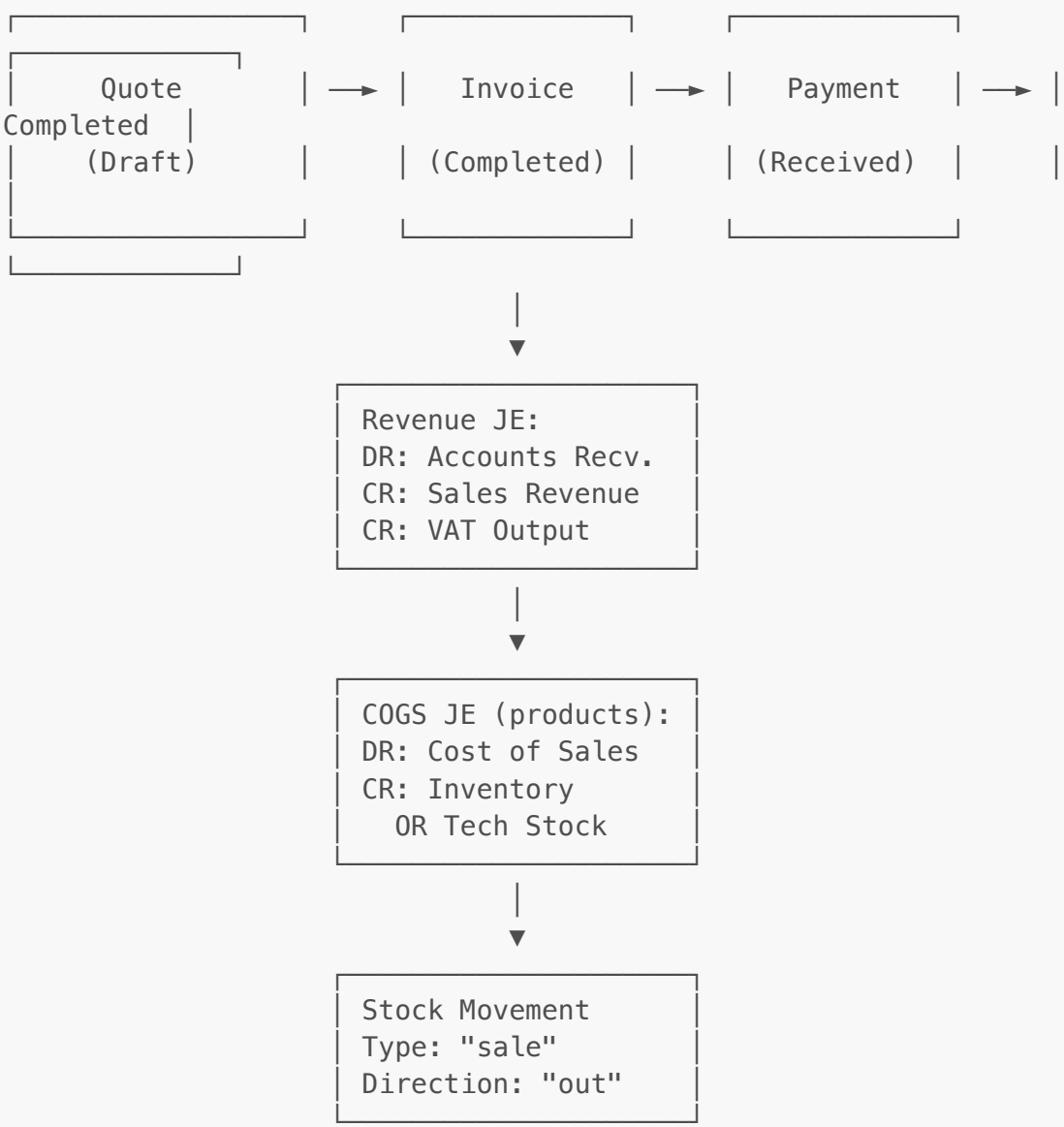
4. TRANSACTION FLOWS

Purchase Flow (Supplier → Us)



```
                        │ Tax Transactions    │
                        │ - VAT Input         │
                        │ - WHT (if applicable)│
                        └─────────────────────┘
```

Sales Flow (Us → Customer)

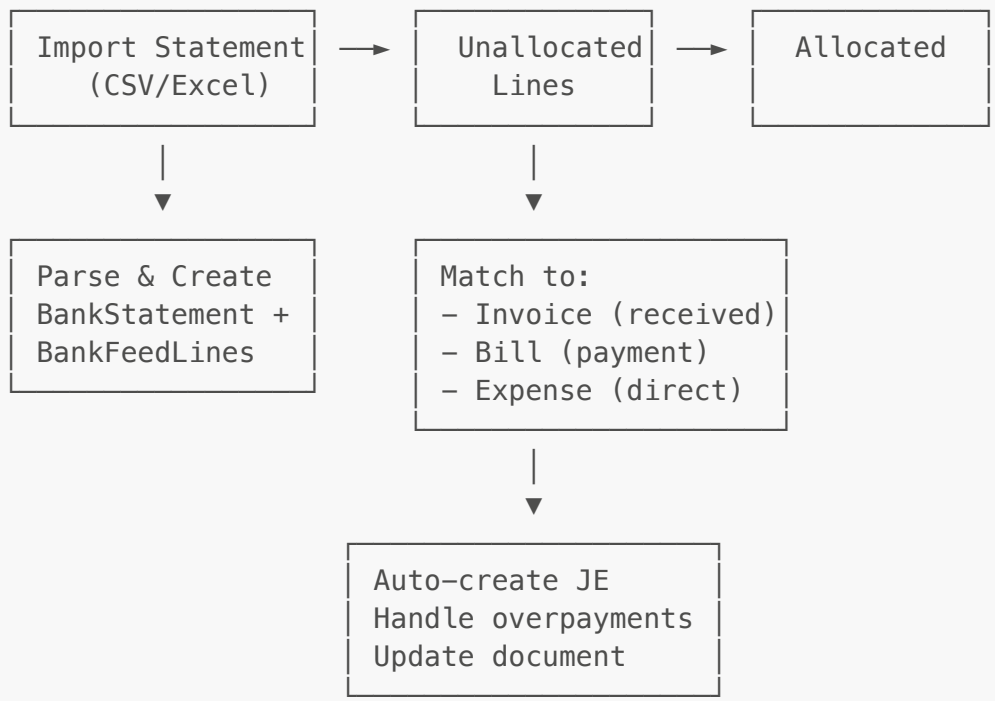


Overpayment Handling (NEW)

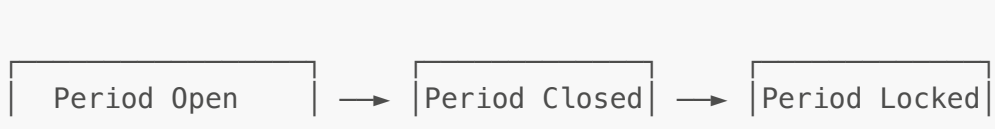
```
┌──────────────────────────────────────────────────────────────────────────────┐
│ When bank payment exceeds document balance:                                  │
│                                                                              │
│                                                                              │
│ CUSTOMER OVERPAYMENT (Invoice):                                              │
```

```
  |   DR: Bank Account           (full bank amount)
  |
  |   CR: Accounts Receivable    (invoice balance only)
  |
  |   CR: Customer Advance       (excess amount - LIABILITY)
  |
  |
  |
  | SUPPLIER OVERPAYMENT (Bill):
  |
  |   DR: Accounts Payable       (bill balance only)
  |
  |   DR: Supplier Advance       (excess amount - ASSET)
  |
  |   CR: Bank Account           (full bank amount)
  |
  |
  |
  | Later, advances can be applied to future invoices/bills
  |
  └──────────────────────────────────────────────────────────────────────────────
┘
```

Bank Feed Allocation Flow (NEW)



Fiscal Period Flow (NEW)



```
│  (Active)       │     │ (Adjustable)│     │ (Permanent) │
└─────────────────┘     └─────────────┘     └─────────────┘
        │                     │                   │
        ▼                     ▼                   ▼
   • New transactions    • Closing JE created  • No changes
   • Documents can       • Revenue/Expense     • Audit trail
     be completed          to Retained Earn.   • Historical
   • Auto-created if     • Statistics calc'd   • Archive
     missing             • Can reopen (Admin)
```

---

5. REMAINING FEATURES TO COMPLETE

5.1 Expense Approval Workflow (Priority: HIGH)

Current State: Schema 90%, Actions 60%, UI 85%

Missing:

```
// Need to complete in expense-actions.js:
// - approveExpense(expenseId, user) - Full approval with JE creation
// - rejectExpense(expenseId, reason, user)
// - payExpense(expenseId, paymentMethod, user) - Separate payment action
```

5.2 Bank Feed Reconciliation (Priority: HIGH)

Current State: Schema 80%, Actions 70%, UI 60%

Missing:

- Auto-matching algorithm (suggestions based on amount/reference)
- Bulk allocation UI
- Reconciliation summary/completion
- Duplicate detection

5.3 Credit Note Actions (Priority: MEDIUM)

Current State: Schema 95%, Actions 50%

Missing:

```
// Need to complete in credit-note-actions.js:
// - approveCreditNote() - Create reversal JE + stock return
// - applyCreditToInvoice() - Apply credit balance to another invoice
// - issueRefund() - Create refund payment
```

5.4 Dashboard Completion (Priority: MEDIUM)

Accountant Dashboard:

- Wire up KPI calculations from journalQueries
- Add bank reconciliation status widget
- Add period-end checklist

Executive Dashboard:

- Create executive-queries.js with high-level metrics
- Revenue trends, cash position, key ratios
- Department performance summary

---

6. DEVELOPMENT PHASES

✅ Phase 1: Core Transaction Completion (COMPLETED)

Task	Status	Notes
Bill server actions	✅ Done	1,306 lines
Bill UI (list, create, detail)	✅ Done	Full components
Expense server actions	🔨 60%	Approval missing
Expense UI	✅ Done	Forms complete
Purchase Order schema + actions	✅ Done	Full workflow
Purchase Order UI	✅ Done	With PDF
Payment schema + actions	✅ Done	991 lines
Fiscal Period validation	✅ Done	Invoice + Bill

✅ Phase 2: Sales Flow (COMPLETED)

Task	Status	Notes
Quote schema	✅ Done	25KB
Quote actions	✅ Done	878 lines
Quote UI + PDF	✅ Done	Full suite
Quote → Invoice conversion	✅ Done	Working
Invoice with salesPerson	✅ Done	Added field
Invoice ↔ StockRequest link	✅ Done	items.relatedRequest

✅ Phase 3: PDF Generation (COMPLETED)

Task	Status	Notes

Task	Status	Notes
Setup @react-pdf/renderer	✅ Done	v4.3.0
Base PDF template/components	✅ Done	7 shared components
Quote PDF	✅ Done	731 lines
Invoice PDF	✅ Done	1,263 lines
Credit Note PDF	✅ Done	Working
Purchase Order PDF	✅ Done	150 lines

🔨 Phase 4: Banking & Reconciliation (IN PROGRESS)

Task	Status	Priority
Bank feed schema	✅ Done	-
Statement import (CSV/Excel)	✅ Done	-
Unallocated lines view	✅ Done	-
Manual allocation UI	✅ Done	-
Overpayment handling	✅ Done	-
Auto-matching suggestions	🔨 50%	🔴 HIGH
Bulk allocation	❌ Not started	🟡 MED
Reconciliation completion	❌ Not started	🟡 MED

🔨 Phase 5: Dashboard & Reports (IN PROGRESS)

Task	Status	Priority
Admin Dashboard	✅ 85%	-
SuperAdmin Dashboard	✅ 80%	-
Employee Dashboard	✅ 80%	-
Accountant Dashboard	🔨 50%	🟡 MED
Executive Dashboard	🔨 30%	🟡 MED
AR/AP Aging Reports	✅ Done	-
VAT/WHT Reports	🔨 60%	🟡 MED
Cash Flow Report	🔨 50%	🟢 LOW

❌ Phase 6: Advanced Features (NOT STARTED)

Task	Priority	Notes

Task	Priority	Notes
Batch operations UI	🟡 MED	Bulk payments, bulk posting
API endpoints	🟡 MED	External integrations
Audit trail viewer	🟢 LOW	Schema ready
Multi-currency support	🟢 LOW	Schema ready
Workflow automation	🟢 LOW	Future

---

7. PDF TEMPLATES

Available Templates

Document	File	Lines	Status
Invoice	`InvoicePDF.jsx`	1,263	✅ Production
Quote	`QuotePDF.jsx`	731	✅ Production
Purchase Order	`PurchaseOrderPDF.jsx`	150	✅ Production
Credit Note	`CreditNotePDF.jsx`	~400	✅ Production
Statement	`StatementPDF.jsx`	~300	✅ Production
Supplier Statement	`SupplierStatementPDF.jsx`	~300	✅ Production

Shared PDF Components

```
/app/components/pdf/
├── templates/          # Document templates
├── components/
│   ├── DocumentHeader.jsx    # Company logo, address
│   ├── DocumentFooter.jsx    # Page numbers, terms
│   ├── LineItemsTable.jsx    # Items table
│   ├── TotalsSection.jsx     # Subtotal, tax, total
│   ├── PartyInfo.jsx         # Customer/Supplier block
│   ├── BankDetails.jsx       # Payment instructions
│   └── NotesSection.jsx      # Terms & conditions
└── styles/
    └── pdfStyles.js          # Shared styles
```

Missing PDF Templates

Document	Priority	Notes
Bill/Payment Voucher	🟡 MED	For supplier payments
Delivery Note	🟢 LOW	Basic template needed

Document	Priority	Notes
Receipt	🟢 LOW	Payment confirmation

8. UTILITY FUNCTIONS REFERENCE

Entry Number Generators

Function	Location	Format	Used By
`generateUniqueEntryNumber(prefix)`	`/lib/utils/server-utils.js`	`JE-{PREFIX}-0001`	All JE creators
`generateMovementNumber()`	StockMovement static	`MOV-DDMMYY-0001`	Stock movements
`generateAdjustmentNumber()`	InventoryAdjustment static	`ADJ-YYYY-0001`	Adjustments
`generateRequestNumber()`	Request actions	`REQ-DDMMYY-001`	Stock requests
`generateBillNumber()`	Bill static	`BILL-{CODE}-YYYYMM-0001`	Bills
`generateInvoiceNumber()`	Invoice actions	`INV-{CODE}-YYYYMM-0001`	Invoices

Journal Entry Prefixes

Prefix	Used For	Example
`SALE`	Invoice revenue	`JE-SALE-0001`
`COGS`	Invoice COGS	`JE-COGS-0001`
`BILL`	Bill approval	`JE-BILL-0001`
`EXP`	Expense	`JE-EXP-0001`
`PAY`	Payment made	`JE-PAY-0001`
`REC`	Payment received	`JE-REC-0001`
`ADV`	Employee advance	`JE-ADV-0001`
`ADJ`	Adjustment	`JE-ADJ-0001`
`REV`	Reversal	`JE-REV-0001`
`CLOSE`	Period closing	`JE-CLOSE-2025-01`

Formatting Functions

Function	Location	Example
`formatCurrency(amount, compact?)`	`/lib/utils.js`	`KES 1,500,000.00` or `KES 1.5M`
`formatRelativeTime(date)`	`/lib/utils.js`	`5m ago`, `2h ago`
`toTitle(string)`	`/lib/utils.js`	`HELLO WORLD` → `Hello World`

9. SYSTEM ACCOUNTS REQUIRED

Account Setup Checklist

System Account	Account Type	Required By	Status
`accounts_receivable`	Asset	Invoice, Payment	✅
`accounts_payable`	Liability	Bill, Payment	✅
`inventory`	Asset	Bill, Invoice, Adjustment	✅
`technician_stock`	Asset	Request fulfillment	✅
`cogs`	Expense	Invoice COGS	✅
`sales_revenue`	Revenue	Invoice	✅
`service_revenue`	Revenue	Invoice (services)	✅
`vat_input`	Asset	Bill, Expense	✅
`vat_output`	Liability	Invoice	✅
`wht_payable`	Liability	Bill	✅
`employee_advance`	Asset	Employee Claims	✅
`employee_payables`	Liability	Employee Claims	✅
`retained_earnings`	Equity	Period closing	✅
`inventory_adjustment`	Expense	Stock Adjustment	✅
`cash`	Asset	Payments	✅
`bank_main`	Asset	Payments	✅
`mpesa`	Asset	Payments	✅
`supplier_advance`	Asset	Overpayments on bills	✅ NEW
`customer_advance`	Liability	Overpayments on invoices	✅ NEW

10. QUICK REFERENCE - WHAT CREATES WHAT

Action	Creates JE?	Creates Movement?	Creates Tax Txn?	Updates Product?

Action	Creates JE?	Creates Movement?	Creates Tax Txn?	Updates Product?
addStock (new)	❌	✅ initial	❌	✅ create
updateStock	❌	❌	❌	✅ update
StockAdjustment.approve()	✅ auto	✅ auto	❌	✅ qty
Bill.approve()	✅ auto	✅ if products	✅ VAT+WHT	✅ qty+cost
Bill.recordPayment()	✅ auto	❌	❌	❌
Invoice.complete()	✅ Revenue+COGS	✅ sale	✅ VAT	✅ qty
Invoice.recordPayment()	✅ auto	❌	❌	❌
Expense.approve()	✅ auto	❌	✅ if VAT	❌
fulfillRequest()	✅ auto	✅ issue	❌	✅ qty
returnItemCheckout()	✅ auto	✅ return	❌	✅ qty
EmployeeClaim.approve()	✅ auto	❌	❌	❌
BankFeed.allocate()	✅ auto	❌	❌	❌
FiscalPeriod.close()	✅ closing	❌	❌	❌

11. CODE METRICS

Server Actions (app/mongodb/actions/)

File	Lines	Status
claim-action.js	3,085	✅ Complete
bill-actions.js	1,306	✅ Complete
purchase-order-actions.js	1,134	✅ Complete
payment-actions.js	991	✅ Complete
company-actions.js	966	✅ Complete
quote-actions.js	878	✅ Complete
party-actions.js	862	✅ Complete
account-actions.js	827	✅ Complete
stock-actions.js	797	✅ Complete
bank-feed-actions.js	752	🔨 WIP
category-actions.js	760	✅ Complete

File	Lines	Status
expense-actions.js	490	🔨 60%
fiscal-period-actions.js	363	✅ Complete
credit-note-actions.js	304	🔨 50%
journal-actions.js	240	✅ Complete
adjustment-actions.js	183	✅ Complete

Query Files (app/mongodb/queries/)

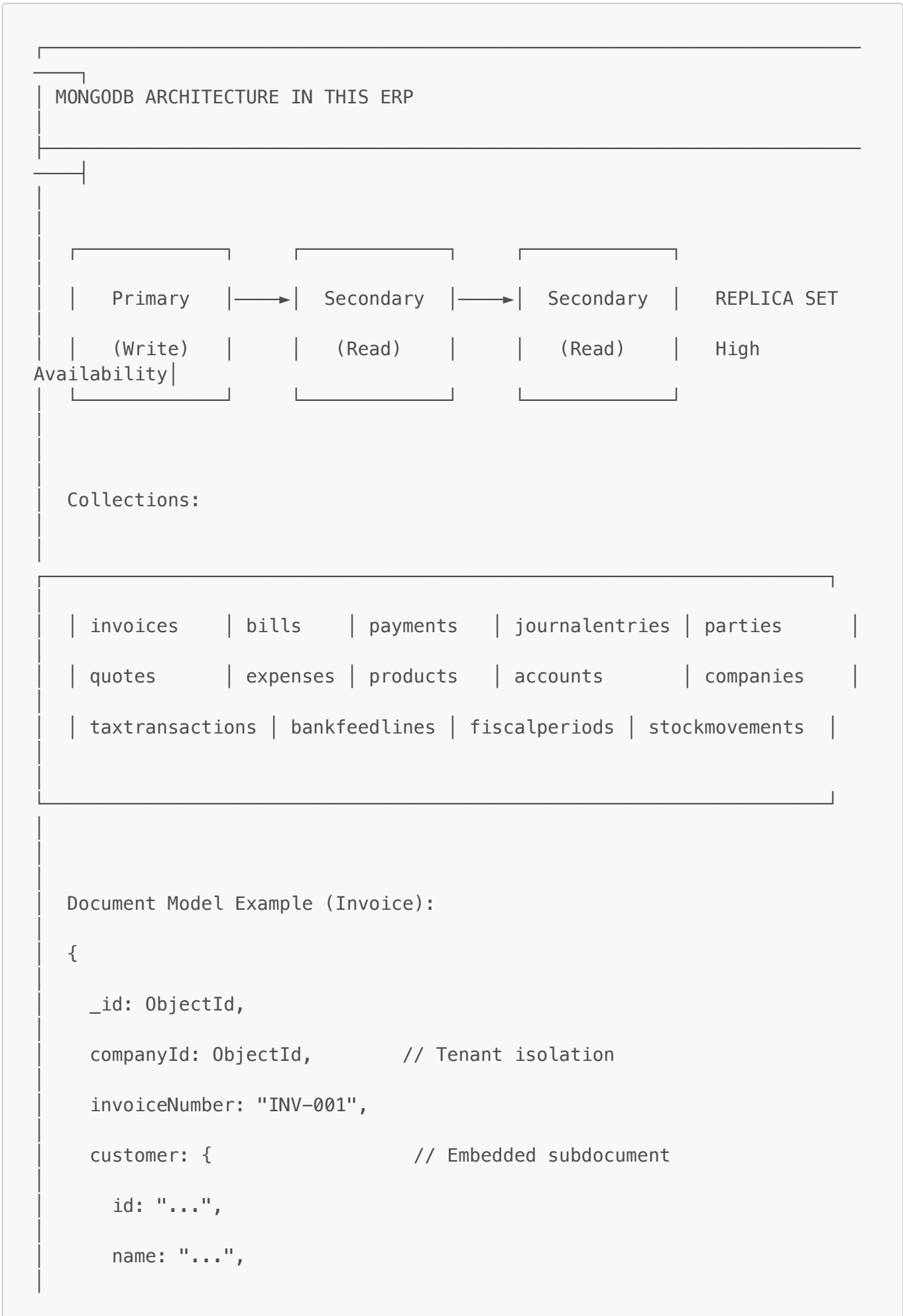
File	Lines	Notes
queries.js	38,649	Master query file
company-queries.js	26,298	Platform metrics
statement-queries.js	25,381	Banking
partyQueries.js	18,519	Customer/Supplier
bank-feed-queries.js	15,379	Bank feed
journalQueries.js	15,472	Accounting
bill-queries.js	14,643	Bills
invoice-queries.js	14,417	Invoices
taxQueries.js	14,163	Tax reporting
accountQueries.js	13,246	Accounts

12. MONGODB ARCHITECTURE FOR ERP

Why MongoDB for ERP Systems

Factor	MongoDB	SQL (Relational)
Schema Evolution	Flexible - add fields without migrations	Rigid - requires ALTER TABLE
Business Objects	Natural document model (Order + Lines + Taxes)	Multiple tables with JOINs
ACID Transactions	✅ Multi-document transactions	✅ Native
Horizontal Scale	Built-in sharding	Complex, often manual
High Availability	Replica sets with auto-failover	Requires additional setup
Developer Experience	Documents match application objects	ORM complexity

MongoDB Core Concepts Used



```
  │      email: "..."
  │
  │    },
  │
  │    items: [                    // Embedded array
  │
  │      { productId, name, qty, price, ... },
  │
  │      { productId, name, qty, price, ... }
  │
  │    ],
  │
  │    amounts: { subtotal, tax, total },
  │
  │    accounting: {               // Related references
  │
  │      revenueJournalEntryId: ObjectId,
  │
  │      cogsJournalEntryId: ObjectId
  │
  │    }
  │
  │  }
  │
  └──────────────────────────────────────────────────────────────────────────
  ─────┘
```

Indexing Strategy

Index Type	Usage	Example
Single Field	Direct lookups	`{ companyId: 1 }`
Compound	Multi-field queries	`{ companyId: 1, status: 1, date: -1 }`
Unique	Enforce uniqueness	`{ companyId: 1, invoiceNumber: 1 }` (unique)
Text	Full-text search	`{ description: "text", reference: "text" }`
TTL	Auto-expire documents	Session tokens, temporary data

Aggregation Framework

Used extensively for reports and analytics:

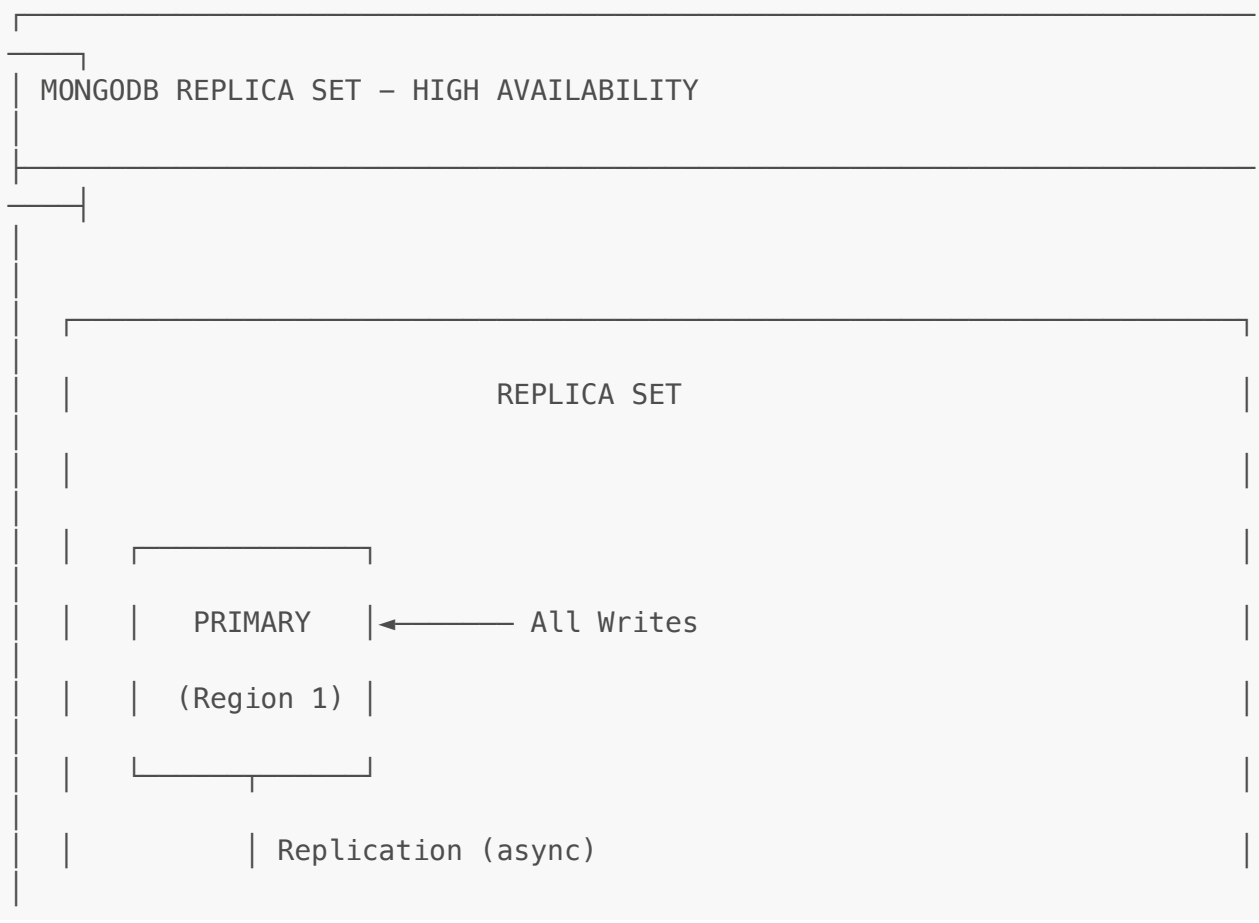
```
// Example: Sales by month with tenant isolation
Invoice.aggregate([
  { $match: { companyId: ObjectId(companyId), status: "completed" } },
  { $group: {
      _id: { $dateToString: { format: "%Y-%m", date: "$invoiceDate" } },
      totalSales: { $sum: "$amounts.total" },
      count: { $sum: 1 }
```

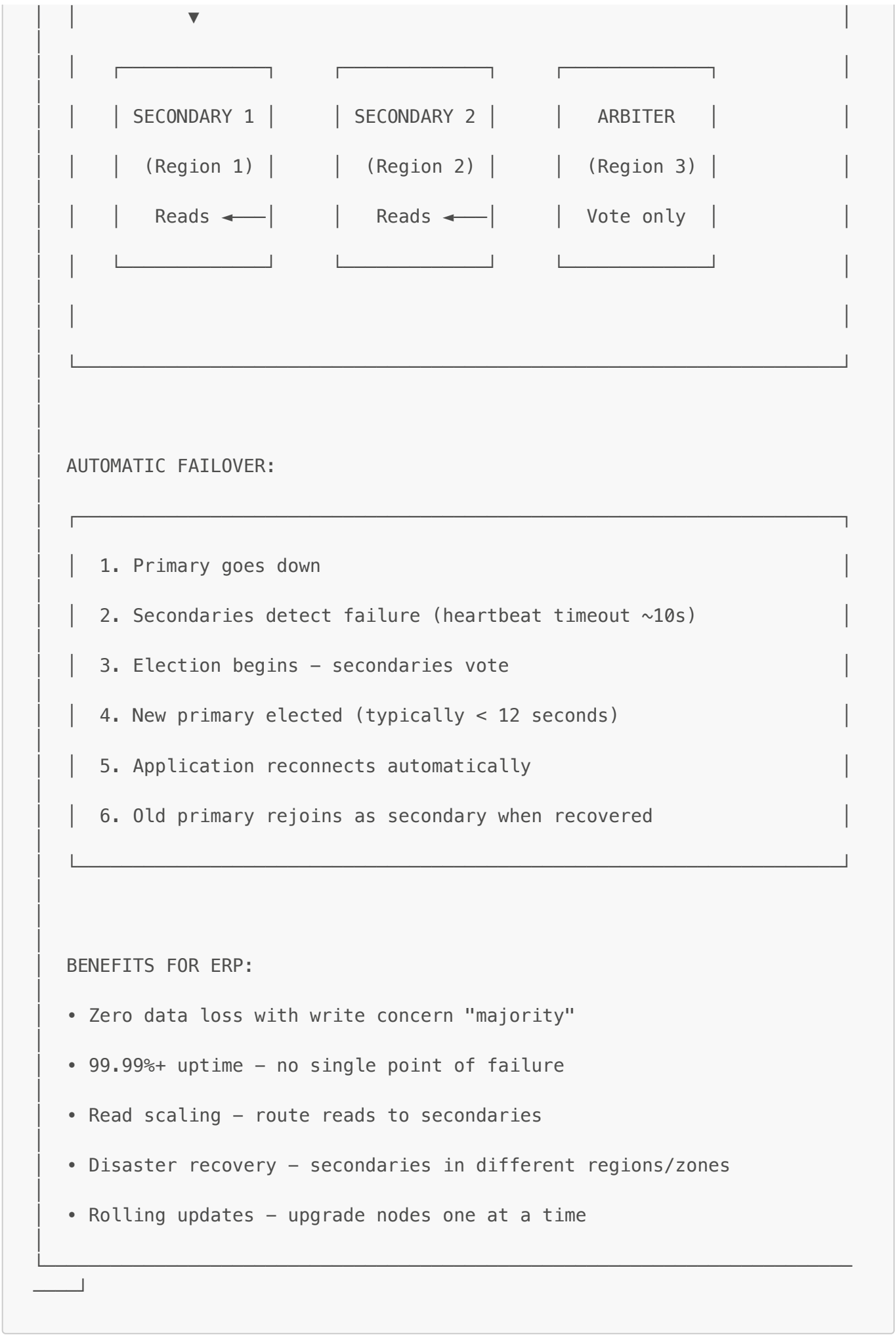
```
  }},
  { $sort: { _id: -1 } }
])
```

Connection Pooling

```
// dbConnect.js pattern - singleton connection
let cached = global.mongoose;
if (!cached) {
  cached = global.mongoose = { conn: null, promise: null };
}

export default async function dbConnect() {
  if (cached.conn) return cached.conn;
  cached.promise = mongoose.connect(MONGODB_URI, {
    maxPoolSize: 10,    // Connection pool size
    serverSelectionTimeoutMS: 5000,
    socketTimeoutMS: 45000,
  });
  cached.conn = await cached.promise;
  return cached.conn;
}
```

High Availability with Replica Sets



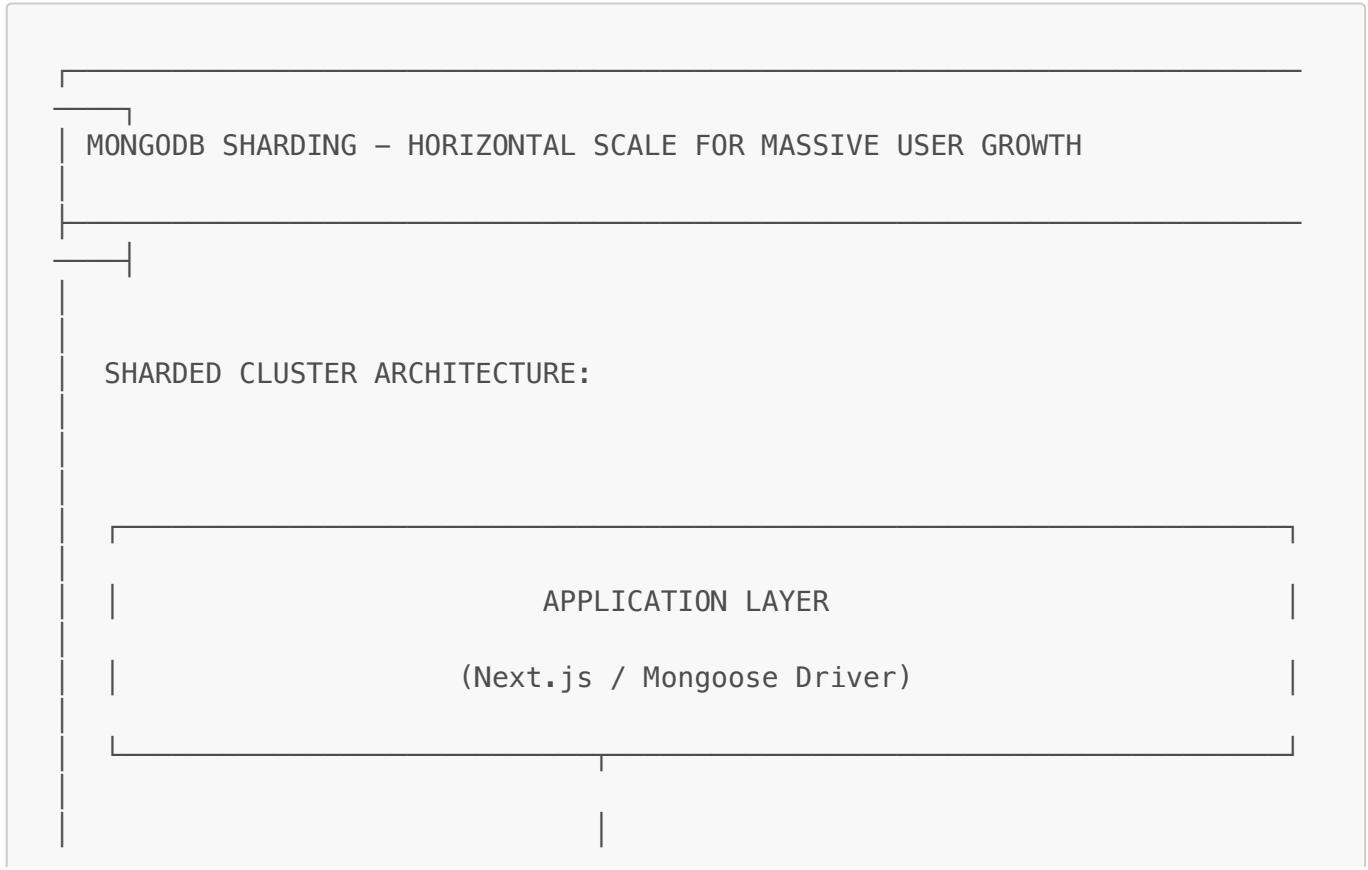

▼
SECONDARY 1
(Region 1)
Reads
SECONDARY 2
(Region 2)
Reads
ARBITER
(Region 3)
Vote only
AUTOMATIC FAILOVER:
1. Primary goes down
2. Secondaries detect failure (heartbeat timeout ~10s)
3. Election begins - secondaries vote
4. New primary elected (typically < 12 seconds)
5. Application reconnects automatically
6. Old primary rejoins as secondary when recovered
BENEFITS FOR ERP:
• Zero data loss with write concern "majority"
• 99.99%+ uptime - no single point of failure
• Read scaling - route reads to secondaries
• Disaster recovery - secondaries in different regions/zones
• Rolling updates - upgrade nodes one at a time

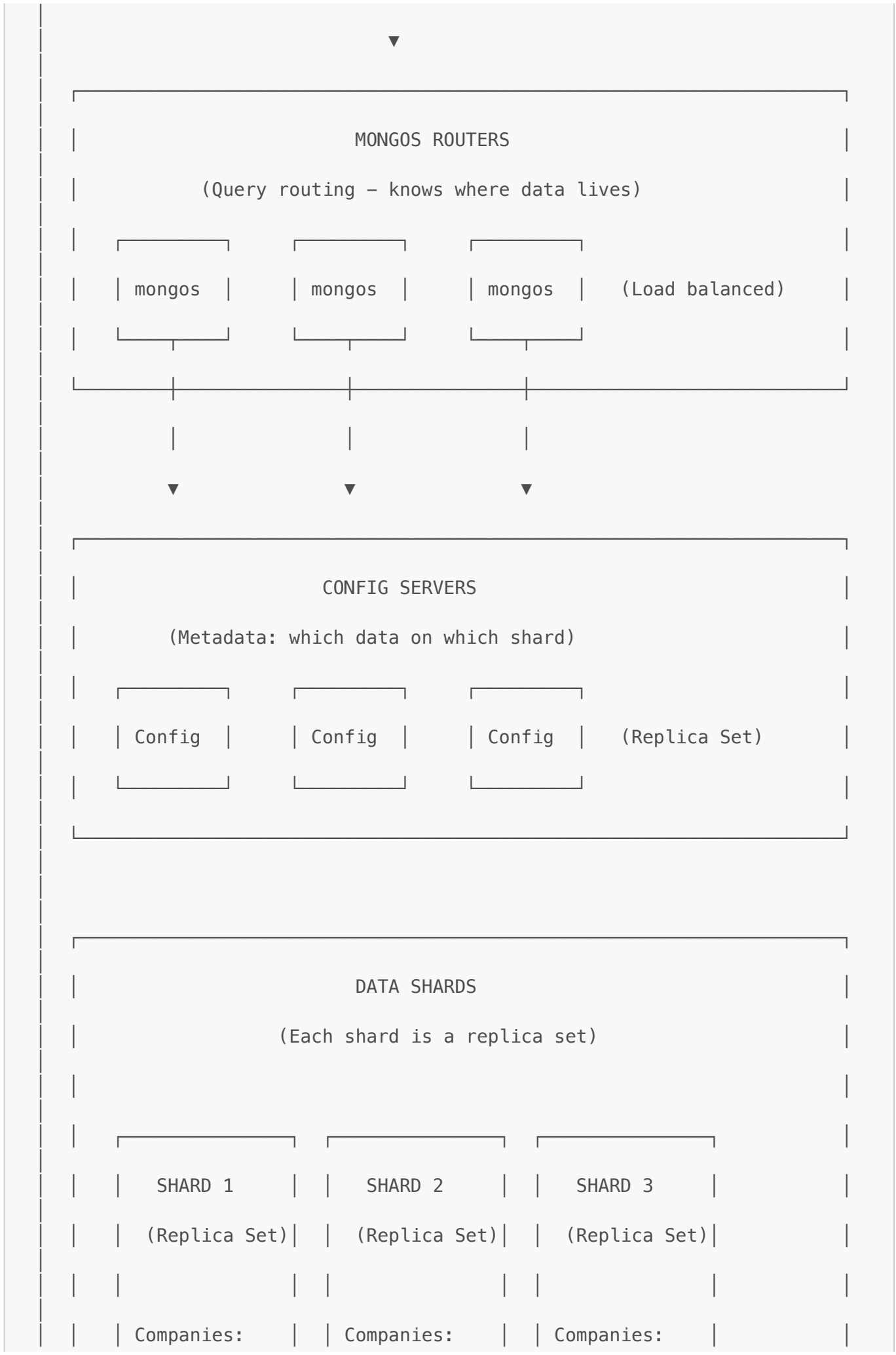
Read Preferences for ERP Workloads:

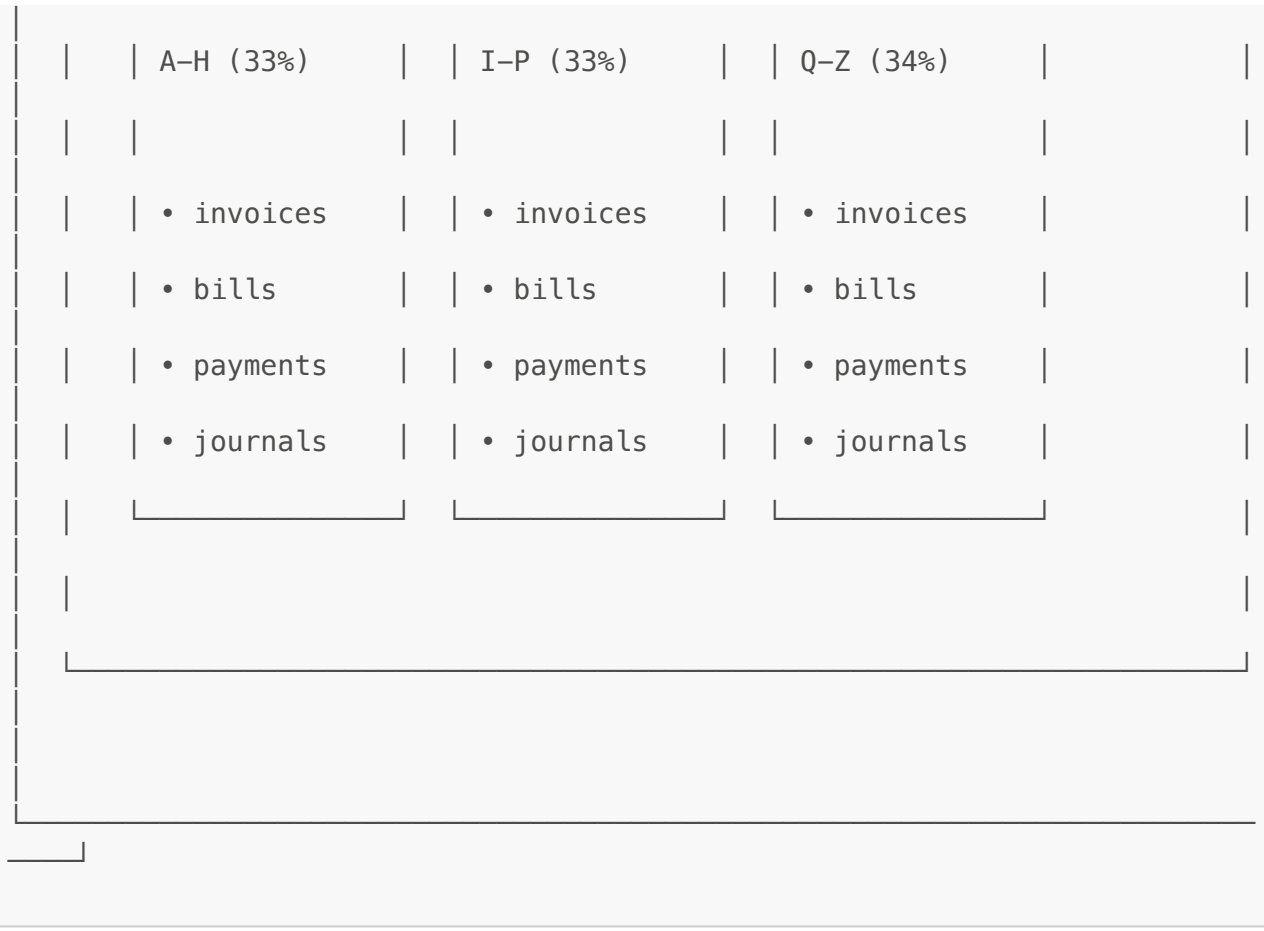
Read Preference	Use Case	Trade-off
`primary` (default)	Financial transactions, real-time data	Highest consistency
`primaryPreferred`	Most reads, fallback if primary down	Good balance
`secondary`	Reports, analytics, dashboards	May read stale data
`secondaryPreferred`	Heavy read workloads	Lower latency
`nearest`	Geo-distributed users	Lowest latency

```
// Connection string with replica set
const MONGODB_URI = "mongodb+srv://user:pass@cluster.mongodb.net/erp?
retryWrites=true&w=majority";

// Read from secondary for reports (eventual consistency OK)
const reportData = await Invoice.find({ status: "completed" })
  .read("secondaryPreferred")
  .lean();

// Write concern for critical financial data
const invoice = await Invoice.create(data, {
  writeConcern: { w: "majority", j: true }  // Wait for majority + journal
});
```

Horizontal Scaling with Sharding




MONGOS ROUTERS
(Query routing – knows where data lives)
mongos
mongos
mongos
(Load balanced)
CONFIG SERVERS
(Metadata: which data on which shard)
Config
Config
Config
(Replica Set)
DATA SHARDS
(Each shard is a replica set)
SHARD 1
(Replica Set)
SHARD 2
(Replica Set)
SHARD 3
(Replica Set)
Companies:
Companies:
Companies:



Shard Key Strategy for Multi-Tenant ERP:

Strategy	Shard Key	Pros	Cons
companyId (Recommended)	`{ companyId: 1 }`	All company data on same shard, efficient queries	Hot spots if one company very large
companyId + date	`{ companyId: 1, createdAt: 1 }`	Spreads large companies, time-based queries efficient	Scatter-gather for company-wide queries
Hashed companyId	`{ companyId: "hashed" }`	Even distribution	Scatter-gather for range queries

```
// Sharding setup for this ERP (MongoDB Atlas or self-managed)

// 1. Enable sharding on database
sh.enableSharding("erp_production")

// 2. Shard collections by companyId (keeps tenant data together)
sh.shardCollection("erp_production.invoices", { companyId: 1 })
sh.shardCollection("erp_production.bills", { companyId: 1 })
sh.shardCollection("erp_production.payments", { companyId: 1 })
sh.shardCollection("erp_production.journalentries", { companyId: 1 })
sh.shardCollection("erp_production.taxtransactions", { companyId: 1 })
sh.shardCollection("erp_production.stockmovements", { companyId: 1 })

// 3. For very high-volume collections, use compound shard key
```

```
sh.shardCollection("erp_production.bankfeedlines", { companyId: 1,
transactionDate: 1 })
```

Why companyId as Shard Key:

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ QUERY ROUTING WITH companyId SHARD KEY                                       │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  TARGETED QUERY (Fast - goes to one shard):                                  │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │  Invoice.find({ companyId: "ABC", status: "completed" })             │    │
│  │                                                                      │    │
│  │  mongos knows companyId "ABC" → Shard 2                              │    │
│  │  Query goes ONLY to Shard 2 → Fast response                          │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
│  SCATTER-GATHER (Slower - goes to all shards):                               │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │  Invoice.find({ status: "completed" })  // No companyId!             │    │
│  │                                                                      │    │
│  │  mongos doesn't know which shard → Query ALL shards                  │    │
│  │  Merge results → Slower response                                     │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
│  THIS IS WHY ALL QUERIES MUST INCLUDE companyId:                             │
│  • getTenantContext() ensures companyId in every query                       │
│  • withTenantScope() adds companyId to all filters                           │
│  • Indexes always start with companyId                                       │
```

```
│
│
│
└──────────────────────────────────────────────────────────────────────────────
─────┘
```

Scaling Milestones

Users/Tenants	Data Volume	Architecture	Notes
< 100 companies	< 10 GB	Single Replica Set (M10-M30)	Current stage
100-1,000 companies	10-100 GB	Replica Set (M40-M60)	Add read replicas
1,000-10,000 companies	100 GB - 1 TB	Start Sharding (2-3 shards)	Shard hot collections
10,000+ companies	1+ TB	Full Sharding (3+ shards)	Add shards as needed
Global	Multi-region	Zone Sharding	Data locality by region

MongoDB Atlas Auto-Scaling (Recommended for Production):

```
// MongoDB Atlas connection - handles scaling automatically
const MONGODB_URI =
"mongodb+srv://user:pass@cluster0.xxxxx.mongodb.net/erp";

// Atlas features used:
// ✅ Auto-scaling compute (M10 → M60 based on load)
// ✅ Auto-scaling storage (grows automatically)
// ✅ Automated backups (point-in-time recovery)
// ✅ Global clusters (multi-region)
// ✅ Performance Advisor (index suggestions)
// ✅ Real-time monitoring
```

Capacity Planning

```
┌──────────────────────────────────────────────────────────────────────────────
─────┐
│ CAPACITY ESTIMATION FOR ERP GROWTH
│
├──────────────────────────────────────────────────────────────────────────────
─────┤
│
│
│  DOCUMENT SIZE ESTIMATES (average):
│
│  • Invoice: ~5 KB (with 10 line items)
│
│  • Bill: ~4 KB
│
```

```
• Payment: ~2 KB
•
• Journal Entry: ~3 KB (with 4 lines)
•
• Tax Transaction: ~1 KB
│
│
PER COMPANY PER MONTH (typical SME):
•
• Invoices: 100 × 5 KB = 500 KB
•
• Bills: 50 × 4 KB = 200 KB
•
• Payments: 150 × 2 KB = 300 KB
•
• Journals: 300 × 3 KB = 900 KB
•
• Tax: 200 × 1 KB = 200 KB
•
─────────────────────────────────
•
Total: ~2 MB/company/month = ~25 MB/company/year
│
│
SCALING PROJECTIONS:
•
• 100 companies × 25 MB = 2.5 GB/year
•
• 1,000 companies × 25 MB = 25 GB/year
•
• 10,000 companies × 25 MB = 250 GB/year
•
• 100,000 companies × 25 MB = 2.5 TB/year
│
│
+ Indexes (~30% overhead)
•
+ Historical data retention
│
│
│
└──────────────────────────────────────────────────────────────────────────────
─────┘
```

13. NEXT.JS & NODE.JS ARCHITECTURE

Node.js Event Loop


NODE.JS EVENT LOOP - SINGLE THREADED, NON-BLOCKING I/O
EVENT LOOP
Timers
setTimeout
Pending
Callbacks
Idle
Prepare
Poll
(I/O)
Check
setImmed.
WHY THIS MATTERS FOR ERP:
• Single thread handles many concurrent requests
• Database queries don't block other requests
• Efficient for I/O-heavy workloads (DB, file, network)
• Low memory footprint per connection
BEST PRACTICES FOLLOWED:
• Avoid blocking the event loop (no sync I/O in request handlers)
• Use async/await for all database operations
• Parallel queries with Promise.all() where possible
• Streaming for large data exports

Next.js 16 App Router Architecture

```
│   ├── layout.js           # Root layout (Server Component)
│   │
│   ├── page.js             # Home page (Server Component)
│   │
│   ├── dashboard/
│   │
│   │   ├── layout.js       # Dashboard layout with sidebar
│   │
│   │   ├── page.tsx        # Dashboard home (Server - fetches data)
│   │
│   │   ├── invoices/
│   │
│   │   │   ├── page.jsx           # List (Server - queries DB)
│   │
│   │   │   ├── [id]/page.jsx      # Detail (Server - single fetch)
│   │
│   │   │   └── components/
│   │
│   │   │       └── InvoiceForm.jsx  # Form ("use client" - interactive)
│   │
│   └── api/                # API Routes (when needed)
│
└──────────────────────────────────────────────────────────────────────────
```

Server-Side Rendering (SSR) vs Static vs Client

Rendering	When	Use Case in ERP
Static (SSG)	Build time	Marketing pages, docs
Server (SSR)	Request time	Dashboard, reports, invoices
Client (CSR)	Browser	Forms, real-time updates
Streaming	Progressive	Large tables, slow queries

React Suspense & Streaming

```jsx
// Pattern used in dashboard pages
import { Suspense } from "react";

export default async function DashboardPage() {
  return (
    <div className="space-y-6">
      {/* Header renders immediately */}
      <h1>Dashboard</h1>

      {/* Stats stream in as they resolve */}
      <Suspense fallback={<StatsSkeleton />}>
        <StatsCards />  {/* async server component */}
      </Suspense>
```

```
      {/* Chart streams independently */}
      <Suspense fallback={<ChartSkeleton />}>
        <SalesChart />  {/* async server component */}
      </Suspense>

      {/* Table streams last (slowest query) */}
      <Suspense fallback={<TableSkeleton />}>
        <RecentInvoices />  {/* async server component */}
      </Suspense>
    </div>
  );
}

// Async Server Component - direct DB access
async function StatsCards() {
  const stats = await getERPDashboardStats();  // Server action
  return <StatsDisplay data={stats} />;
}
```

Benefits of Suspense in ERP:

- Users see content progressively (no blank screens)
- Slow database queries don't block entire page
- Better perceived performance
- Graceful loading states

Server Actions (Form Handling)

```
// Server Action - runs on server, called from client
"use server";

export async function createInvoice(formData) {
  const { companyId } = await getTenantContext();

  // Validate
  const validated = invoiceSchema.parse(formData);

  // Create in database (direct access, no API needed)
  const invoice = await Invoice.create({
    ...validated,
    companyId,
  });

  // Revalidate cache
  revalidatePath("/dashboard/invoices");

  return { success: true, id: invoice._id };
}
```

14. SECURITY & VULNERABILITY PREVENTION

OWASP Top 10 Mitigations

Vulnerability	Prevention in This ERP
Injection (SQL/NoSQL)	Mongoose ODM with parameterized queries, no raw queries
Broken Auth	NextAuth.js with secure session management
Sensitive Data	Server Components (secrets never reach client)
XXE	No XML processing, JSON only
Broken Access	Multi-tenancy isolation, role-based checks
Misconfig	Environment variables, no secrets in code
XSS	React auto-escapes, CSP headers
Insecure Deserialization	JSON.parse with validation, Zod schemas
Vulnerable Components	Regular npm audit, dependabot
Logging	Audit trails on sensitive operations

Authentication & Authorization

```
│     │   • /dashboard/* requires authentication                          │
│     │   • Redirect to /login if no session                              │
│     ├──────────────────────────────────────────────────────────────────┤
│     │ Layer 2: Page-level (role check)                                  │
│     │   const session = await auth();                                   │
│     │   if (session.user.role !== "Admin") redirect("/unauthorized");   │
│     ├──────────────────────────────────────────────────────────────────┤
│     │ Layer 3: Action-level (tenant isolation)                          │
│     │   const { companyId } = await getTenantContext();                 │
│     │   // All queries scoped to companyId                              │
│     └──────────────────────────────────────────────────────────────────┘
│
└──────────────────────────────────────────────────────────────────────────┘
```

Input Validation (Zod Schemas)

```typescript
// Every server action validates input
import { z } from "zod";

const invoiceItemSchema = z.object({
  productId: z.string().min(1),
  name: z.string().min(1).max(200),
  quantity: z.number().positive(),
  unitPrice: z.number().nonnegative(),
  taxRate: z.number().min(0).max(100).optional(),
});

const createInvoiceSchema = z.object({
  customerId: z.string().min(1),
  invoiceDate: z.coerce.date(),
  dueDate: z.coerce.date(),
  items: z.array(invoiceItemSchema).min(1),
  notes: z.string().max(2000).optional(),
});

// In server action
export async function createInvoice(formData) {
  const validated = createInvoiceSchema.safeParse(formData);
```

```
  if (!validated.success) {
    return { error: validated.error.flatten() };
  }
  // Proceed with validated.data
}
```

NoSQL Injection Prevention

```
// ❌ VULNERABLE - Never do this
const user = await User.findOne({
  email: req.body.email,           // Could be { $gt: "" }
  password: req.body.password      // Could be { $ne: null }
});

// ✅ SAFE - Mongoose with explicit types
const user = await User.findOne({
  email: String(req.body.email),   // Coerce to string
  password: String(req.body.password)
});

// ✅ SAFER - Zod validation first
const { email, password } = loginSchema.parse(req.body);
const user = await User.findOne({ email });
```

Environment Variables & Secrets

```
# .env.local (never committed)
MONGODB_URI=mongodb+srv://...
NEXTAUTH_SECRET=random-32-char-string
NEXTAUTH_URL=https://app.example.com

# Accessed only in server code
// ✅ Server Component or Server Action
const dbUri = process.env.MONGODB_URI;

// ❌ Client Component - will be undefined
// (Next.js strips non-NEXT_PUBLIC_ vars from client)
```

Rate Limiting & Protection

```
// Middleware example for API protection
import { Ratelimit } from "@upstash/ratelimit";

const ratelimit = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(10, "10 s"),  // 10 requests per 10
```

```
seconds
});

export async function middleware(request) {
  const ip = request.ip ?? "127.0.0.1";
  const { success } = await ratelimit.limit(ip);

  if (!success) {
    return new Response("Too Many Requests", { status: 429 });
  }
}
```

Audit Trail Pattern

```
// All sensitive operations log audit trail
invoiceSchema.methods.complete = async function(completedBy) {
  // ... business logic ...

  // Audit trail
  this.history.push({
    action: "completed",
    performedBy: {
      name: completedBy.name,
      id: completedBy.id,
    },
    performedAt: new Date(),
    details: {
      previousStatus: this.status,
      newStatus: "completed",
    },
  });

  await this.save();
};
```

---

15. SUMMARY

Overall Completion: ~80%

Production Ready (95%+):

- Invoicing with PDF
- Quotes with PDF
- Purchase Orders with PDF
- Bills
- Payments
- Stock Management
- Journal Entries

- Chart of Accounts
- Employee Claims
- Fiscal Periods

Near Complete (70-90%):

- Expenses (approval workflow needed)
- Credit Notes (apply/refund actions needed)
- Bank Feed (reconciliation completion needed)
- Reports (some need refinement)
- Dashboards (Accountant/Executive incomplete)

In Development (50-70%):

- Full bank reconciliation
- Tax compliance reports
- Advanced analytics

Not Started:

- API for integrations
- Batch operations
- Multi-currency
- Audit trail viewer

END OF DOCUMENT
